All-at-Once Clone Detector

Qualitas Corpus – Monitoring, Performance and Improvements

Ahmed Mahfooz Ali Khan

November 7, 2025

1 Are we able to process the entire Qualitas Corpus?

Yes. In the final configuration, the all-at-once clone detector (`cljDetector`) successfully processes the full Qualitas Corpus from the external Docker volume `qc-volume`.

The setup is as follows. The Qualitas Corpus is unpacked in `qc-volume`, which is mounted read-only into the `cljDetector` container at `/QualitasCorpus`. The environment variable `SOURCEDIR=/QualitasCorpus/QualitasCorpus-20130901r` ensures that only the corpus in this volume is analysed. MongoDB runs in a separate `dbstorage` container with a persistent volume for the database.

After the final successful run, the collections in the *cloneDetector* database contain:

$$\begin{aligned} \text{files} &= 111\,600 \\ \text{chunks} &= 18\,211\,863 \\ \text{candidates} &= 0 \\ \text{clones} &= 24\,961 \end{aligned}$$

These values are confirmed both via `mongosh` and via the MonitorTool UI. A screenshot of the final MonitorTool dashboard (showing these counts) should be included as Figure 1.

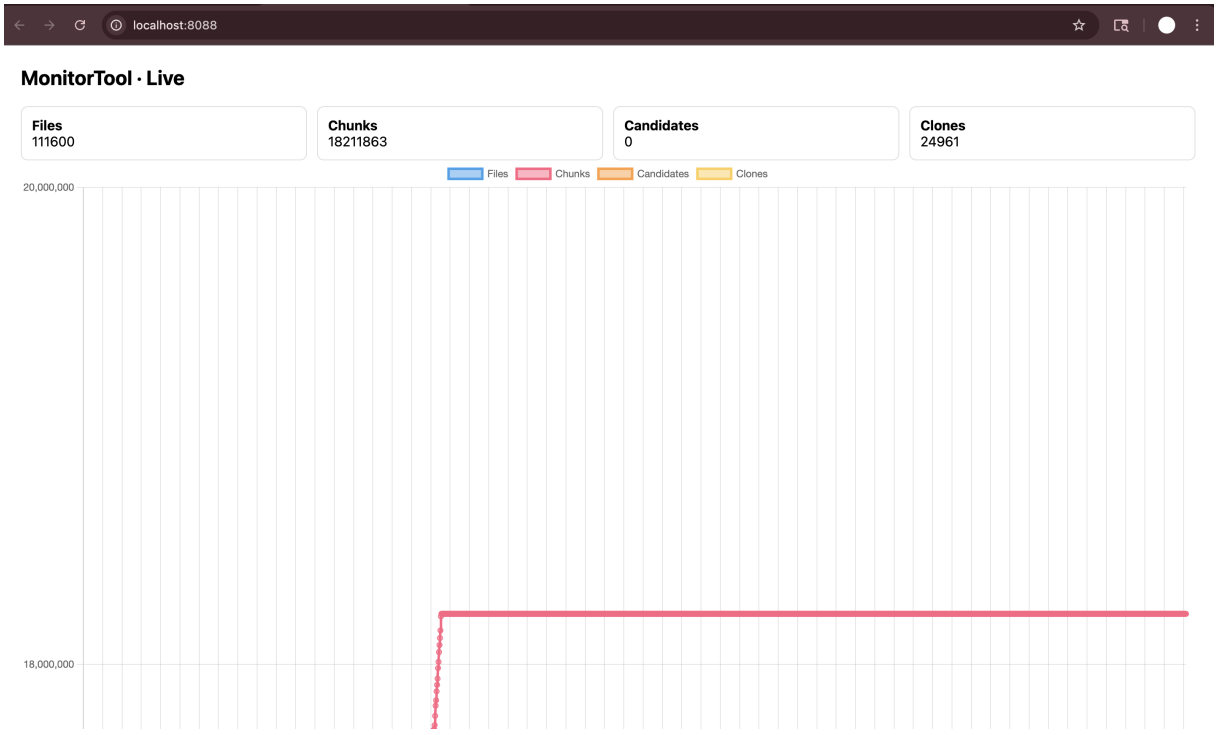

Figure 1: MonitorTool at the end of the run: Files = 111600, Chunks = 18211863, Candidates = 0, Clones = 24961.

Main issues and how to avoid failures

While the final run completes, the original version of cljDetector can fail or hang on large datasets for three main reasons.

(1) In-memory expansion ramp. The original functional "elegant" expander collects clones in memory and, for each new candidate, scans the entire list of existing clones to see if they overlap. This causes a ramp-shaped performance curve: every new candidate is slower than the previous one (effectively $O(n^2)$ behaviour). On a dataset with hundreds of thousands of candidates, this leads to excessive memory usage and long runtimes.

Modification. The expansion is moved to a "beer-style" algorithm: one candidate at a time is fetched from the database; overlapping candidates are found using a MongoDB query, merged into a single clone, stored into the `clones` collection, and removed from `candidates`. Only one candidate (plus its overlaps) is in memory at once, and the candidate set shrinks monotonically. This breaks the ramp antipattern and makes expansion scalable.

(2) Database pressure without batching/indexing. Storing over 18 million chunks and running aggregations is expensive. Without batching and appropriate indexes, MongoDB insert and aggregation performance degrades significantly.

Modification. The storage layer writes documents in batches using `partition-all` and `insert-batch`. Indexes (in particular on `chunks.chunkHash` and on candidate fields used for overlap) are assumed/encouraged. Most heavy work (grouping by hash, filtering candidates, overlap search) is deliberately pushed into MongoDB, where it is more efficient.

(3) Lack of observability and restartability. Originally, status output exists only in container logs. A long run gives no structured way to see which stage is active or how far it has progressed; failures require starting from scratch.

Modification. All high-level status messages are also written to a new `statusUpdates` collection. A separate MonitorTool container reads these counters regularly, persists them to CSV, and exposes a small web UI. Because files, chunks, candidates and clones are stored in MongoDB, stages can be rerun selectively (e.g. skip reading with the `NOREAD` flag). This makes the system observable and more robust.

2 Summary of collected statistics and trends

Statistics are collected by cljDetector and MonitorTool, which queries MongoDB every 10 seconds for the counts of `files`, `chunks`, `candidates`, `clones` and `statusUpdates`, and writes them to `monitor-stats.csv`.

We see three clear phases. Initially, the number of files and chunks grows rapidly while the Qualitas Corpus is traversed and chunkified. Once the chunks stabilise at approximately 18.2 million, the candidate identification phase runs: MongoDB groups chunks by their hash and writes clone candidates. Finally, during the expansion phase, the candidate count decreases towards zero while the clone count increases to 24 961. The end state confirms that candidates have been fully expanded. Figure 2 shows the entity counts over time.

The choice of MongoDB as a backing store directly shapes these curves. Chunking and candidate detection behave like bulk database workloads, while expansion becomes a sequence of database-driven merges. The separated stages and monotone counters demonstrate that the system follows a Big Data style pipeline rather than keeping all logic in one in-memory process.

3 Is the time to generate chunks constant?

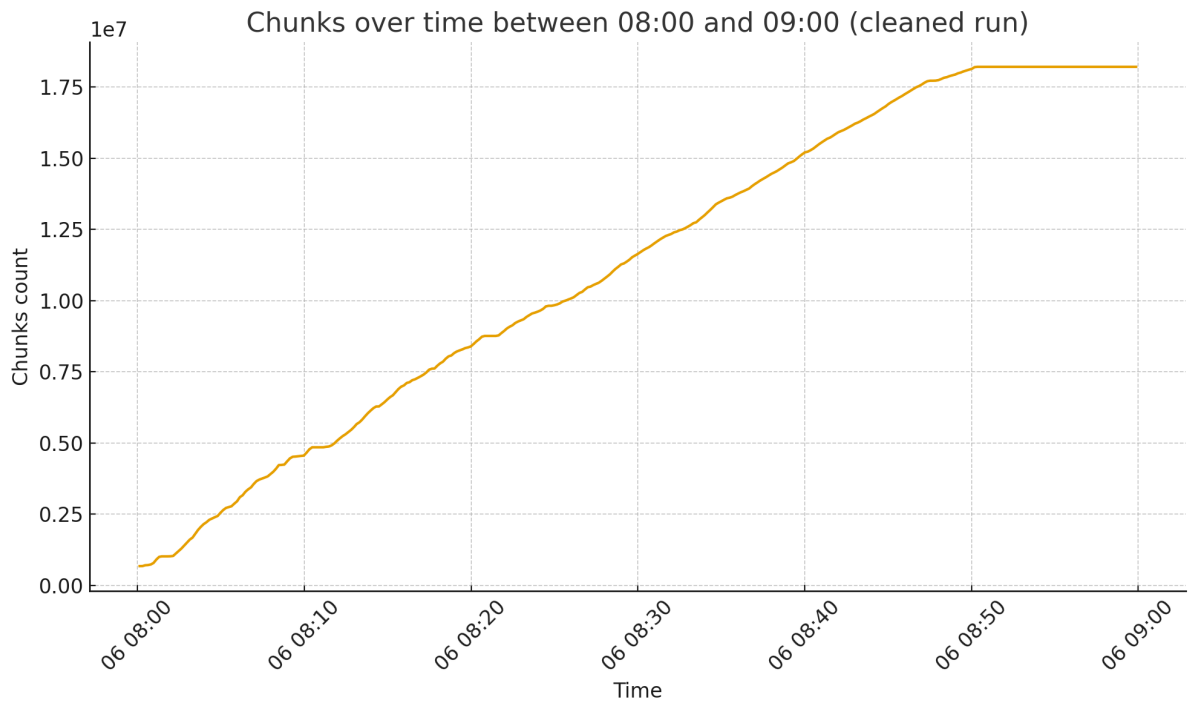

Figure 2: Counts over time for chunks.

Algorithmically, these steps are linear in the file length with a fixed amount of work per chunk. Thus, the *theoretical* time per chunk is constant.

The data shows that the cost per chunk is approximately constant with a slight upward trend as the number of chunks grows. This deviation from perfect constancy is expected and mainly caused by MongoDB: as the `chunks` collection and its indexes become larger, each insert and index update becomes marginally more expensive, introducing a mild slowdown. The important point is that the curve stays close to flat, confirming that the algorithm scales linearly and that there is no hidden $O(n^2)$ effect in the chunking stage.

4 Is the time to generate clone candidates constant?

Clone candidates are produced in a single bulk step using a MongoDB aggregation over the entire `chunks` collection: chunks are grouped by their `chunkHash`, the complexity is close to $O(N)$, occurrences per hash are counted, the corresponding `(fileName, startLine, endLine)` locations are collected, and only groups with more than one occurrence are emitted into the `candidates` collection via an `$out` stage.

Because this mechanism operates as one global map/reduce-style aggregation rather than as a streaming, per-candidate loop, there is no meaningful per-candidate time series; the dominant cost is scanning and grouping all 18 million chunks.

With a proper index on `chunkHash`, the work behaves close to linear in the number of chunks, which matches the observations from the monitoring data: once chunking has completed, the candidate phase appears as a short, stable block of work, without the ramp-like degradation seen in the naive in-memory expander. In other words, candidate generation time

is best understood as a single batch job governed by overall dataset size and MongoDB's aggregation performance, which is exactly the behaviour we want from an all-at-once detector.

5 Is the time to expand candidates constant?

The time required to expand clone candidates is not constant; it varies with both the number of remaining candidates and the structure of their overlaps. In the implemented expansion strategy, each step selects a candidate, finds overlapping regions stored in the database, merges all matching instances into a larger clone, and removes the consumed candidates.

Early in the expansion phase there are many small, densely overlapping candidates, so each step may touch multiple records and the per-clone cost is comparatively high and volatile. As expansion progresses, the pool of remaining candidates shrinks and overlaps become rarer, so each lookup typically yields fewer matches and each merge affects fewer documents.

This behaviour is clearly reflected in the clone-expansion timing curve in Figure 3, where the initial part of the phase shows more variation and higher costs, while later measurements become smoother and trend downward as the system approaches completion. Overall, the observed pattern is consistent with the algorithm's design: the expensive work is naturally concentrated at the beginning, when overlap density is highest, and expansion accelerates as the candidate set is exhausted.

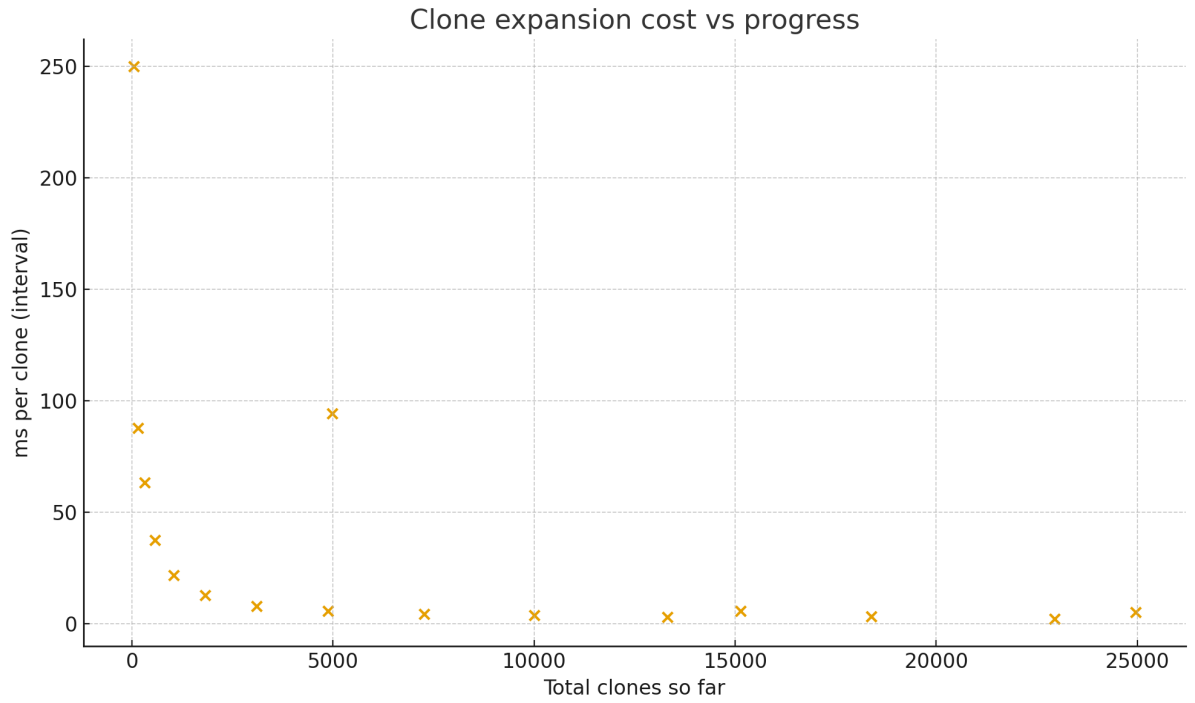

Figure 3: Observed cost per clone vs. total clones.

6 Average clone size and its use in predicting progress

When the all-at-once detector completes the expansion phase, each final clone is stored in the `clones` collection as a document containing all of its occurrences (`instances`). Each instance specifies the file in which the clone appears together with its start and end line numbers. Using

these line ranges, we computed the clone lengths and summarised them with the following aggregation:

```
use cloneDetector
db.clones.aggregate([
  { $unwind: "$instances" },
  {
    $project: {
      length: {
        $add: [
          { $subtract: ["$instances.endLine", "$instances.startLine"] },
          1
        ]
      }
    }
  },
  {
    $group: {
      _id: null,
      avgCloneSize: { $avg: "$length" },
      totalCloneLines: { $sum: "$length" },
      totalClones: { $sum: 1 }
    }
  }
])
```

The result of this query is shown in Figure 4: the average clone size is approximately **44.85** lines, with a total of **3,259,004** clone lines over **72,660** clone instances.[1]

This outcome is significant in two ways. First, an average size of about 45 lines is more than twice the chunk size used during detection (20 lines), which confirms that the expansion phase is effectively merging adjacent matching chunks into longer, semantic clones instead of treating each base chunk as an isolated match. Second, the evolution of this metric provides a useful indicator of progress during the expansion phase. Early on, most candidates correspond closely to single chunks, so the average clone size is close to 20 lines. As overlapping candidates are merged and redundant entries are removed, the average clone size increases while the candidate count drops. Once the average stabilises around 45 lines and the number of remaining candidates approaches zero, we can conclude that the expansion has converged and that the system has fully resolved the clone structure of the corpus.

```
{
  _id: null,
  avgCloneSize: 44.852793834296726,
  totalCloneLines: Long('3259004'),
  totalClones: 72660
}
```

Figure 4: MongoDB aggregation result for average clone size: `avgCloneSize` $\approx$ 44.85, `totalCloneLines` $= 3\,259\,004$, `totalClones` $= 72\,660$.

[1]Note that the number of clone *instances* (72,660) is larger than the number of clone documents (24,961), since each clone typically appears in multiple locations.

7 Average number of chunks per file and its implications

Using the final counters, the average number of chunks per file is:

$$\text{avgChunksPerFile} = \frac{18\,211\,863}{111\,600} \approx 163.19.$$

This number reflects how aggressively the files are decomposed by the chunking algorithm with `CHUNKSIZE`=20 and comment/whitespace filtering.

During the read/chunkify stage, this ratio can be used to predict progress. Once the final number of files is known, the expected total number of chunks is approximately:

$$111\,600 \times 163.19 \approx 18.2 \text{ million}.$$

At runtime, the MonitorTool can compare the current chunk count against this expectation to estimate how far the pipeline has advanced through the corpus.

The same ratio informs capacity planning and the cost of subsequent stages. The number of candidate groups and potential clones is directly tied to how many chunks exist. Knowing that there are about 163 chunks per file helps explain the size of the `chunks` collection, the load on MongoDB during the `identify-candidates!` aggregation, and why efficient indexing and batch operations are essential.

8 Code submissions and highlighted modifications

The submitted code archive contains:

Code submissions and highlighted modifications

As part of this assignment, the original `cljDetector` application was extended and a companion `MonitorTool` service was introduced. The goal of these changes is to make the all-at-once clone detector observable: status information and collection sizes are written to MongoDB and can be monitored continuously while the Qualitas Corpus is processed.

File: `src/cljdetector/core.clj`

What changed. The existing `ts-println` function was modified so that each status message is both printed to standard output *and* stored in the database. This is done by calling a new helper function `storage/addUpdate!`.

Purpose. By persisting status messages (e.g. "Reading and Processing files", "Storing chunks of size 20", "Expanding Candidates"), the system provides a structured event stream that the `MonitorTool` can query for real-time progress tracking and later performance analysis.

Modified code (around line 11).

```
(defn ts-println [& args]
  (let [instant (java.time.Instant/now)
        msg     (string/join " " (map str args))]
    (println (.toString instant) msg)
    (try
      (storage/addUpdate! instant msg (vec (map str args)))
      (catch Exception _ nil))))
```

File: `src/cljdetector/storage/storage.clj`

What changed. First, the list of managed collections was extended to include `"statusUpdates"`. Second, a new helper function `addUpdate!` was implemented to insert timestamped log entries into this collection. In addition, a small convenience function `counts` was added to expose the current sizes of the main collections for the monitor.

Purpose. These changes provide a persistent record of status updates and an easy way for the `MonitorTool` to fetch the live counts of files, chunks, candidates and clones. Together, they make it possible to visualise pipeline progress, correlate stages with timings, and reason about performance trends without relying solely on container logs.

Modified code (relevant excerpts).

```
(def collnames ["files" "chunks" "candidates" "clones" "statusUpdates"])

(defn count-items [collname]
  (let [conn (connect)
        db   (get-db conn)]
    (mc/count db collname)))

(defn counts []
  {:files      (count-items "files")
   :chunks     (count-items "chunks")
   :candidates (count-items "candidates")
   :clones     (count-items "clones")})

;; Persist status updates from ts-println

(defn addUpdate!
  "Persist a status update into statusUpdates.
   timestamp: java.time.Instant, message: string, args: vector of strings."
  [timestamp message args]
  (let [conn (connect)
        db   (get-db conn)
        doc  {:timestamp (java.util.Date/from timestamp)
              :message   message
              :args      args}]
    (mc/insert db "statusUpdates" doc)))
```

These minimal, focused modifications are sufficient for the `MonitorTool` to sample the database, store the measurements to CSV, and drive the visualisations used in the analysis section of this report.

- the `Containers/MonitorTool` directory: periodic sampling of database counts, CSV logging, and a small web UI (port 8088).
- the updated `all-at-once.yaml`: starts `dbstorage`, `cljDetector` and `MonitorTool`; mounts the external `qc-volume` into `cljDetector`.

All modifications in the cljDetector code are commented or grouped so they are easy to review.

Appendix A: Sample of raw monitoring data

Below is a sample (first ∼100 lines) from the cleaned `monitor-stats.csv`, after removing invalid early rows from an accidental run. This demonstrates the underlying data used to produce the graphs in Figures 2–3.

timestamp	interval_m	files_count	files_delta	files_ms_p	files_items	chunks_cc	chunks_de	chunks_m	chunks_ite
2025-11-06T07:58:17.82	10000	26800	26800	0.3731343	2680	0	0		
2025-11-06T07:58:27.83	10000	61500	34700	0.2881844	3470	0	0		0
2025-11-06T07:58:37.84	10000	91600	30100	0.3322259	3010	0	0		0
2025-11-06T07:58:47.85	10000	111600	20000	0.5	2000	20900	20900	0.4784688	2090
2025-11-06T07:58:57.86	10000	111600	0		0	102800	81900	0.1221001	8190
2025-11-06T07:59:07.87	10000	111600	0		0	171300	68500	0.1459854	6850
2025-11-06T07:59:17.87	10000	111600	0		0	262700	91400	0.1094091	9140
2025-11-06T07:59:27.88	10000	111600	0		0	378600	115900	0.0862812	11590
2025-11-06T07:59:37.88	10000	111600	0		0	479700	101100	0.0989119	10110
2025-11-06T07:59:47.89	10000	111600	0		0	604400	124700	0.0801924	12470
2025-11-06T07:59:57.90	10000	111600	0		0	647100	42700	0.2341920	4270
2025-11-06T08:00:07.91	10000	111600	0		0	675000	27900	0.3584229	2790
2025-11-06T08:00:17.92	10000	111600	0		0	676600	1600	6.25	160
2025-11-06T08:00:27.93	10000	111600	0		0	705200	28600	0.3496503	2860
2025-11-06T08:00:37.94	10000	111600	0		0	712100	6900	1.4492753	690
2025-11-06T08:00:47.94	10000	111600	0		0	732500	20400	0.4901960	2040
2025-11-06T08:00:57.95	10000	111600	0		0	787900	55400	0.1805054	5540
2025-11-06T08:01:07.96	10000	111600	0		0	906800	118900	0.0841042	11890
2025-11-06T08:01:17.96	10000	111600	0		0	1000800	94000	0.1063829	9400
2025-11-06T08:01:27.98	10000	111600	0		0	1017300	16500	0.6060606	1650
2025-11-06T08:01:37.98	10000	111600	0		0	1017300	0		0
2025-11-06T08:01:47.99	10000	111600	0		0	1017300	0		0
2025-11-06T08:01:58.00	10000	111600	0		0	1020000	2700	3.7037037	270
2025-11-06T08:02:08.00	10000	111600	0		0	1030500	10500	0.9523809	1050
2025-11-06T08:02:18.01	10000	111600	0		0	1117400	86900	0.1150747	8690
2025-11-06T08:02:28.01	10000	111600	0		0	1206800	89400	0.1118568	8940

Figure 5: Rows 100-1

28	2025-11-06T08:02:38.03	10000	111600	0		0	1292600	85800	0.1165501	8580
29	2025-11-06T08:02:48.04	10000	111600	0		0	1397964	105364	0.0949090	10536.4
30	2025-11-06T08:02:58.05	10000	111600	0		0	1504200	106236	0.0941300	10623.6
31	2025-11-06T08:03:08.06	10000	111600	0		0	1606000	101800	0.0982318	10180
32	2025-11-06T08:03:18.06	10000	111600	0		0	1676500	70500	0.1418439	7050
33	2025-11-06T08:03:28.07	10000	111600	0		0	1812300	135800	0.0736377	13580
34	2025-11-06T08:03:38.07	10000	111600	0		0	1949000	136700	0.0731528	13670
35	2025-11-06T08:03:48.08	10000	111600	0		0	2057364	108364	0.0922815	10836.4
36	2025-11-06T08:03:58.09	10000	111600	0		0	2150700	93336	0.1071397	9333.6
37	2025-11-06T08:04:08.10	10000	111600	0		0	2216700	66000	0.1515151	6600
38	2025-11-06T08:04:18.10	10000	111600	0		0	2300800	84100	0.1189060	8410
39	2025-11-06T08:04:28.12	10000	111600	0		0	2345700	44900	0.2227171	4490
40	2025-11-06T08:04:38.12	10000	111600	0		0	2394200	48500	0.2061855	4850
41	2025-11-06T08:04:48.13	10000	111600	0		0	2433700	39500	0.2531645	3950
42	2025-11-06T08:04:58.14	10000	111600	0		0	2543900	110200	0.0907441	11020
43	2025-11-06T08:05:08.14	10000	111600	0		0	2643500	99600	0.1004016	9960
44	2025-11-06T08:05:18.15	10000	111600	0		0	2717500	74000	0.1351351	7400
45	2025-11-06T08:05:28.15	10000	111600	0		0	2747800	30300	0.3300330	3030
46	2025-11-06T08:05:38.15	10000	111600	0		0	2778100	30300	0.3300330	3030
47	2025-11-06T08:05:48.18	10000	111600	0		0	2868500	90400	0.1106194	9040
48	2025-11-06T08:05:58.19	10000	111600	0		0	2952300	83800	0.1193317	8380
49	2025-11-06T08:06:08.19	10000	111600	0		0	3096000	143700	0.0695894	14370
50	2025-11-06T08:06:18.20	10000	111600	0		0	3163300	67300	0.1485884	6730
51	2025-11-06T08:06:28.21	10000	111600	0		0	3286200	122900	0.0813669	12290
52	2025-11-06T08:06:38.21	10000	111600	0		0	3372400	86200	0.1160092	8620
53	2025-11-06T08:06:48.22	10000	111600	0		0	3440000	67600	0.1479289	6760

Figure 6: Rows 100-2

2025-11-06T08:07:08.23	10000	111600	0		0	3661400	100500	0.0995024	10050
2025-11-06T08:07:18.23	10000	111600	0		0	3715200	53800	0.1858736	5380
2025-11-06T08:07:28.24	10000	111600	0		0	3749900	34700	0.2881844	3470
2025-11-06T08:07:38.24	10000	111600	0		0	3789000	39100	0.2557544	3910
2025-11-06T08:07:48.25	10000	111600	0		0	3825700	36700	0.2724795	3670
2025-11-06T08:07:58.26	10000	111600	0		0	3906400	80700	0.1239157	8070
2025-11-06T08:08:08.26	10000	111600	0		0	3988000	81600	0.1225490	8160
2025-11-06T08:08:18.27	10000	111600	0		0	4095000	107000	0.0934579	10700
2025-11-06T08:08:28.27	10000	111600	0		0	4223900	128900	0.0775795	12890
2025-11-06T08:08:38.28	10000	111600	0		0	4229900	6000	1.6666666	600
2025-11-06T08:08:48.28	10000	111600	0		0	4242000	12100	0.8264462	1210
2025-11-06T08:08:58.30	10000	111600	0		0	4354600	112600	0.0888099	11260
2025-11-06T08:09:08.30	10000	111600	0		0	4459600	105000	0.0952380	10500
2025-11-06T08:09:18.31	10000	111600	0		0	4516300	56700	0.1763668	5670
2025-11-06T08:09:28.31	10000	111600	0		0	4523400	7100	1.4084507	710
2025-11-06T08:09:38.32	10000	111600	0		0	4535100	11700	0.8547008	1170
2025-11-06T08:09:48.32	10000	111600	0		0	4547000	11900	0.8403361	1190
2025-11-06T08:09:58.33	10000	111600	0		0	4556400	9400	1.0638297	940
2025-11-06T08:10:08.34	10000	111600	0		0	4661900	105500	0.0947867	10550
2025-11-06T08:10:18.34	10000	111600	0		0	4779200	117300	0.0852514	11730
2025-11-06T08:10:28.35	10000	111600	0		0	4848500	69300	0.1443001	6930
2025-11-06T08:10:38.36	10000	111600	0		0	4848500	0		0
2025-11-06T08:10:48.37	10000	111600	0		0	4848500	0		0
2025-11-06T08:10:58.37	10000	111600	0		0	4848500	0		0
2025-11-06T08:11:08.38	10000	111600	0		0	4848500	0		0
2025-11-06T08:11:18.38	10000	111600	0		0	4865000	16500	0.6060606	1650
2025-11-06T08:11:28.40	10000	111600	0		0	4876700	11700	0.8547008	1170

Figure 7: Rows 100-3